SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: CSA16 COURSE CODE: Data Warehousing and Data Mining

---

COURSE OUTCOME COVERED

CO5: Apply association rule mining techniques to discover interesting relationships and patterns within large datasets
Assessment Tool Weightage: 35%

QUESTIONS: 05 **Total Marks:50**

Annexure A

Constraint-Based Problem Solving

Criteria	Problem Understanding *(2.5)*	Constraint Analysis *(2.5)*	Solution Development *(2.5)*	Application of Concepts *(1.5)*	Justification *(1)*	*Total(10)*
Weightage	*25%*	25%	25%	15%	10%	*100%*
Q1						
Q2						
Q3						
Q4						
Q5						

Code of Conduct:

I A. LOKESH KUMAR (192524157) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A.LK

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Annexure A

Constraint-Based Problem Solving (6 Marks) (BL4–BL5) – Reframed with New Scenarios

Q1. Dataset: Hospital Patient Records

PID	Symptoms Observed
P1	Fever, Cough
P2	Fever, Headache
P3	Cough, Headache
P4	Fever, Cough, Headache
P5	Fever, Fatigue

Question:
Apply association rule mining with minimum support = 50% and confidence = 70%, under the constraint that all generated rules must include **"Fever"**. Identify the valid association rules and justify why they satisfy the given constraints. *(BL4 – 1.2 Marks)*

Q2. Dataset: Online Learning Platform

Student ID	Courses Enrolled
S1	Python, SQL
S2	Python, Data Science
S3	SQL, Machine Learning
S4	Python, SQL, Data Science
S5	Python, Machine Learning

Question:
Using the dataset, generate frequent itemsets with minimum support = 40%, applying the constraint that each itemset must contain **at least two courses**. Explain how the constraint helps in pruning candidate itemsets during mining. *(BL4 – 1.2 Marks)*

Q3. Dataset: Library Borrowing Records

Transaction ID	Books Borrowed
T1	Data Mining, Python Programming
T2	Data Mining, Database Systems
T3	Python Programming, Database Systems
T4	Data Mining, Python Programming, Database Systems
T5	Data Mining, Artificial Intelligence

Question:
Apply the Apriori algorithm with the constraint that the **maximum itemset size is 3**. Identify the frequent itemsets and explain how the constraint reduces computational complexity during candidate generation. *(BL5 – 1.2 Marks)*

Q4. Dataset: University Course Hierarchy

Student ID	Courses Taken
S1	Engineering, Computer Science, Machine Learning
S2	Engineering, Information Technology
S3	Engineering, Computer Science
S4	Engineering, Computer Science, Machine Learning
S5	Engineering, Information Technology

Question:
Apply multilevel association rule mining with the constraint that rules should be generated only at the **"Computer Science"** level. Explain how the course hierarchy affects the discovery of association rules. *(BL5 – 1.2 Marks)*

Q5. Dataset: Insurance Customer Records

CID	Age Group	Income Level	Policy Purchased
C1	Young	High	Health Insurance
C2	Middle	Medium	Vehicle Insurance
C3	Young	Low	Travel Insurance
C4	Senior	High	Health Insurance
C5	Young	Medium	Vehicle Insurance

Question:
Apply multidimensional association rule mining with the constraint **Age Group = "Young"**. Identify meaningful patterns involving income level and policy type, and explain how these patterns can support targeted marketing strategies. *(BL4 – 1.2 Marks)*

1.

The dataset records symptoms observed for five patients. With minimum support = 50% and minimum confidence = 70%, and the constraint that every generated rule must include the item "Fever", the first step is to compute the support of every relevant item and itemset before checking which candidate rules can satisfy both thresholds simultaneously.

Step 1: Support of Individual Symptoms (Total transactions = 5)

- Fever: appears in P1, P2, P4, P5 → 4/5 = 80%
- Cough: appears in P1, P3, P4 → 3/5 = 60%
- Headache: appears in P2, P3, P4 → 3/5 = 60%
- Fatigue: appears in P5 → 1/5 = 20%

Step 2: Support of 2-Itemsets Containing Fever

- {Fever, Cough}: P1, P4 → 2/5 = 40%
- {Fever, Headache}: P2, P4 → 2/5 = 40%
- {Fever, Fatigue}: P5 → 1/5 = 20%

Step 3: Applying the Minimum Support Threshold (50%)

Since minimum support = 50% on 5 transactions requires a support count of at least 2.5, meaning at least 3 patients must share the itemset, none of the 2-itemsets containing Fever reach this threshold — {Fever, Cough} and {Fever, Headache} each occur in only 2 of 5 patients (40%), and {Fever, Fatigue} occurs in only 1 (20%). Consequently, none of these itemsets qualify as frequent under the stated support constraint, and no association rule built on them can be generated at this stage.

Step 4: Confidence Check for Completeness (Even at Relaxed Support)

- Confidence(Fever → Cough) = support({Fever,Cough}) / support(Fever) = 2/4 = 50%
- Confidence(Fever → Headache) = 2/4 = 50%
- Confidence(Cough → Fever) = 2/3 = 66.7%
- Confidence(Headache → Fever) = 2/3 = 66.7%

Even if the support constraint were relaxed to 40% purely to check whether these candidate rules could satisfy the confidence requirement, none of them reach the required 70% confidence — the highest is 66.7%. This confirms that the failure to generate rules is not an artefact of the support threshold alone; the confidence threshold independently rules out every Fever-containing candidate as well.

Conclusion and Justification

Under the stated constraints — minimum support 50%, minimum confidence 70%, and mandatory inclusion of "Fever" — no valid association rule can be generated from this dataset. This is justified because every 2-itemset containing Fever falls below the 2.5-transaction support threshold, and even the strongest candidate rules (Cough → Fever and Headache → Fever) only reach 66.7% confidence, short of the required 70%. This outcome is itself a valid and important analytical result: it demonstrates that Fever, despite being the most frequent individual symptom (80% support), does not co-occur strongly enough with any single other symptom to support a statistically reliable association rule at the thresholds specified, highlighting the importance of jointly satisfying both support and confidence constraints rather than relying on the popularity of an individual item.

2.

The dataset records course enrolments for five students. With minimum support = 40% (a support count of at least 2 out of 5 transactions) and the constraint that every reported itemset must contain at least two courses, the frequent itemsets are derived by first computing individual course frequencies (needed internally for the Apriori join step) and then evaluating multi-course combinations.

Step 1: Support of Individual Courses (Internal Apriori Step)

- Python: S1, S2, S4, S5 → 4/5 = 80%
- SQL: S1, S3, S4 → 3/5 = 60%
- Data Science: S2, S4 → 2/5 = 40%
- Machine Learning: S3, S5 → 2/5 = 40%

Step 2: Candidate 2-Itemsets and Their Support

- {Python, SQL}: S1, S4 → 2/5 = 40% — meets threshold
- {Python, Data Science}: S2, S4 → 2/5 = 40% — meets threshold
- {Python, Machine Learning}: S5 → 1/5 = 20% — below threshold
- {SQL, Data Science}: S4 → 1/5 = 20% — below threshold
- {SQL, Machine Learning}: S3 → 1/5 = 20% — below threshold
- {Data Science, Machine Learning}: none → 0% — below threshold

Step 3: Candidate 3-Itemsets

By the Apriori anti-monotone property, a 3-itemset can only be frequent if all of its 2-item subsets are frequent. The only 3-itemset worth testing is {Python, SQL, Data Science}, but its subset {SQL, Data Science} is not frequent (20%), so it is pruned without needing to count its support directly. Counting it directly for verification: it occurs only in S4 → 1/5 = 20%, confirming it is indeed infrequent.

Frequent Itemsets Satisfying the Constraint (≥ 2 courses, support ≥ 40%)

- {Python, SQL} — 40% support
- {Python, Data Science} — 40% support

How the Constraint Helps in Pruning

The "at least two courses" constraint has two pruning effects. First, it removes single-item results from the reported output stage, focusing the search directly on itemset combinations that are business-relevant (co-enrolment patterns) rather than individual course popularity. Second, and more importantly for Apriori's efficiency, once the algorithm establishes which 1-itemsets are frequent, the constraint combined with the anti-monotone property means any 2-itemset built from an infrequent 1-itemset can be discarded immediately without counting — for example, since no course besides the four listed appears frequently, the candidate generation step never needs to consider combinations involving rare or non-existent courses, keeping the candidate set small. This illustrates how a simple cardinality constraint, layered on top of the standard Apriori pruning logic, further reduces the number of itemsets that must be counted against the transaction database.

3.

The dataset records books borrowed across five library transactions. The question does not state an explicit minimum support percentage, so for demonstration a minimum support of 40% (a support count of at least 2 out of 5 transactions) is assumed, consistent with the threshold used in the similar Q2 scenario. The stated constraint is that the maximum itemset size explored by Apriori is capped at 3.

Step 1: Support of Individual Books (Level 1)

- Data Mining: T1, T2, T4, T5 → 4/5 = 80%
- Python Programming: T1, T3, T4 → 3/5 = 60%
- Database Systems: T2, T3, T4 → 3/5 = 60%
- Artificial Intelligence: T5 → 1/5 = 20% — below threshold, pruned

Step 2: Candidate 2-Itemsets (Level 2, Built Only from Frequent 1-Itemsets)

- {Data Mining, Python Programming}: T1, T4 → 2/5 = 40% — frequent
- {Data Mining, Database Systems}: T2, T4 → 2/5 = 40% — frequent
- {Python Programming, Database Systems}: T3, T4 → 2/5 = 40% — frequent

Step 3: Candidate 3-Itemsets (Level 3 — the Maximum Allowed by the Constraint)

The only candidate 3-itemset formed by joining the frequent 2-itemsets is {Data Mining, Python Programming, Database Systems}. It occurs only in T4 → 1/5 = 20%, which is below the 40% threshold, so it is not frequent. Since the constraint caps itemset size at 3, Apriori stops here regardless of this outcome — no 4-itemsets are generated or evaluated.

Final Frequent Itemsets

- Size 1: {Data Mining} (80%), {Python Programming} (60%), {Database Systems} (60%)
- Size 2: {Data Mining, Python Programming} (40%), {Data Mining, Database Systems} (40%), {Python Programming, Database Systems} (40%)
- Size 3: none (the only candidate, at 20% support, fails the threshold)

How the Maximum-Itemset-Size Constraint Reduces Computational Complexity

Apriori's candidate generation grows combinatorially with itemset size — without a cap, the algorithm would continue joining level-3 itemsets into level-4 candidates, then level-5, and so on, even in cases where doing so yields no additional frequent patterns (as happened here, since the only level-3 candidate was already infrequent). By fixing the maximum itemset size at 3, the algorithm avoids the join and prune steps entirely for level 4 and beyond, which in a larger dataset with many items and longer transactions would otherwise require generating and support-counting a very large number of high-order candidate itemsets. This constraint is especially valuable in real library systems with thousands of titles, where unconstrained Apriori could face significant candidate-set explosion at higher levels despite most of those candidates ultimately being infrequent or practically uninteresting.

4.

The dataset records courses taken by five students, structured across an implicit concept hierarchy: Engineering sits at the general (top) level, Computer Science and Information Technology sit at a more specific (mid) level as branches of Engineering, and Machine Learning sits at an even more specific (lower) level as a specialisation typically associated with Computer Science. The constraint requires that rules be generated only at the "Computer Science" level rather than at the trivial, overly general Engineering level.

Step 1: Support of Items Across the Hierarchy

- Engineering (top level): S1–S5 → 5/5 = 100%
- Computer Science (mid level): S1, S3, S4 → 3/5 = 60%
- Information Technology (mid level): S2, S5 → 2/5 = 40%
- Machine Learning (lower level, specialisation of Computer Science): S1, S4 → 2/5 = 40%

Step 2: Why Engineering-Level Rules Are Excluded by the Constraint

Because Engineering appears in 100% of transactions, any rule of the form "X → Engineering" would trivially have 100% confidence for every X, and any rule "Engineering → Y" would have confidence equal to Y's own support. These rules are technically valid but analytically uninteresting, since they simply restate that Engineering is the common ancestor of every course path in the hierarchy. This is precisely the redundancy that multilevel association mining is designed to avoid by restricting analysis to a more specific level of the hierarchy — in this case, Computer Science.

Step 3: Rules Generated at the Computer Science Level

- {Computer Science, Machine Learning}: S1, S4 → 2/5 = 40% support
- Confidence(Computer Science → Machine Learning) = 2/3 = 66.7%
- Confidence(Machine Learning → Computer Science) = 2/2 = 100%

These two rules are meaningful and specific to the Computer Science level: the first indicates that roughly two-thirds of Computer Science students in this dataset also take Machine Learning, while the second shows that, within this dataset, every student who takes Machine Learning is also enrolled in Computer Science — a non-trivial, actionable pattern that would have been hidden if the analysis stopped at the generic Engineering level.

How the Course Hierarchy Affects Rule Discovery

The hierarchy determines both which rules are trivial and which minimum support thresholds are realistic at each level. At the top (Engineering) level, support is artificially inflated because the item is an ancestor of virtually every transaction, making the minimum support threshold nearly meaningless as a filter. As the analysis descends to more specific levels, item supports naturally shrink (Computer Science at 60%, Machine Learning at 40%), which is why multilevel mining typically applies progressively reduced minimum support thresholds at lower levels rather than reusing the same threshold across the whole hierarchy. By anchoring the analysis at the Computer Science level, the constraint filters out redundant, ancestor-derived rules and surfaces the specific Computer Science–Machine Learning relationship that is of real analytical value to an academic department planning course offerings or advising students.

5.

The dataset records insurance customers across three dimensions — Age Group, Income Level, and Policy Purchased. The constraint restricts the analysis to the multidimensional slice where Age Group = "Young", after which the task is to identify meaningful patterns linking Income Level and Policy Purchased within that slice.

Step 1: Applying the Age Group = "Young" Constraint

Filtering the five customer records to only those with Age Group = "Young" leaves three records: C1 (High income, Health Insurance), C3 (Low income, Travel Insurance), and C5 (Medium income, Vehicle Insurance). This filtered subset is the multidimensional "slice" on which all subsequent pattern discovery is performed, consistent with how multidimensional association mining first restricts the search space along one or more dimensions before mining relationships among the remaining dimensions.

Step 2: Income–Policy Combinations Within the Young Segment

- Young ∧ High Income → Health Insurance: 1 of 3 Young customers (33% support within the segment), 100% confidence within this income bracket
- Young ∧ Low Income → Travel Insurance: 1 of 3 (33% support), 100% confidence within this income bracket
- Young ∧ Medium Income → Vehicle Insurance: 1 of 3 (33% support), 100% confidence within this income bracket

Interpreting the Pattern

Although each individual combination has low absolute support due to the small size of the Young segment, every income bracket observed among Young customers maps to a distinct, non-overlapping policy type, and each mapping holds with 100% confidence within the filtered segment — no Young customer in the dataset deviates from their income bracket's associated policy. This is a meaningful multidimensional pattern precisely because it is consistent, even though the sample is small: it suggests that within the Young age group, policy preference is strongly differentiated by income level rather than being uniform across the segment.

Supporting Targeted Marketing Strategies

- Young, high-income customers can be targeted with Health Insurance offers, potentially bundled with wellness or long-term investment-linked products that appeal to financially established younger customers.
- Young, low-income customers can be targeted with affordably priced Travel Insurance, possibly promoted through budget-conscious messaging and flexible, short-term coverage options.
- Young, medium-income customers can be targeted with Vehicle Insurance campaigns, timed around common life events for this segment such as purchasing a first car.

- Because these patterns are specific to the Young segment, marketing channels can also be tailored — for example, digital and social-media advertising, which tends to reach younger demographics more effectively than traditional channels used for older segments.

Conclusion

By constraining the multidimensional analysis to Age Group = "Young" before examining the Income Level–Policy Purchased relationship, the mining process surfaces a clear, income-differentiated pattern of policy preference within that age segment. Even though the absolute support of each pattern is limited by the small number of Young customers in the dataset, the perfect within-segment confidence of each income–policy mapping makes these patterns directly actionable for designing targeted, income-specific marketing campaigns aimed at young customers.